4190.428 001 Introduction to Machine Learning



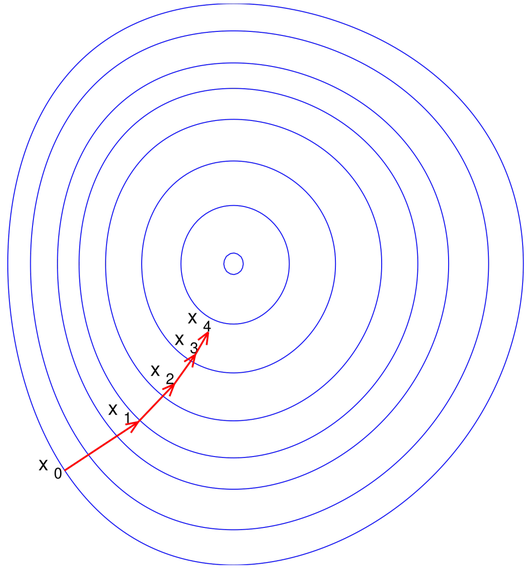
Gradient Descent

Jaesik Park
CSE & IPAI, SNU
jaesik.park@snu.ac.kr

How can we find the optimal parameter?

- Sometimes, we need to find the optimal $\Theta^* = \arg\min_{\Theta} J(\Theta)$
- We can think $J(\Theta)$ as a surface over Θ:
  - We need to find the lowest point on that surface
  - Thus, we need to find which **direction** the “graph hill” goes down most **steeply** and take a small step in that direction





One Dimension Case

- Assume $\Theta \in \mathbb{R}$ and we know both $J(\Theta), J'(\Theta)$. We have to specify
  1. the initial value for parameter Θ
  2. a step-size η, and an
  3. accuracy parameter ε

Algorithm 14: 1D-Gradient-Descent$(\Theta_{\text{init}}, \eta, f, f', \epsilon)$

$\Theta^{(0)} = \Theta_{\text{init}}$
$t = 0$
while $|f(\Theta^{(t)}) - f(\Theta^{(t-1)})| > \epsilon$ **do**
    $t = t + 1$
    $\Theta^{(t)} = \Theta^{(t-1)} - \eta f'(\Theta^{(t-1)})$
return $\Theta^{(t)}$

- This Algorithm stops when the change in the function is sufficiently small

One Dimension Case

- Possible Stopping Criteria
  - Stop after a fixed number of iterations, i.e. when $t = T$
  - Stop when the change in the value of parameter Θ is sufficiently small

$$|\Theta^{(t)} - \Theta^{(t-1)}| < \epsilon$$

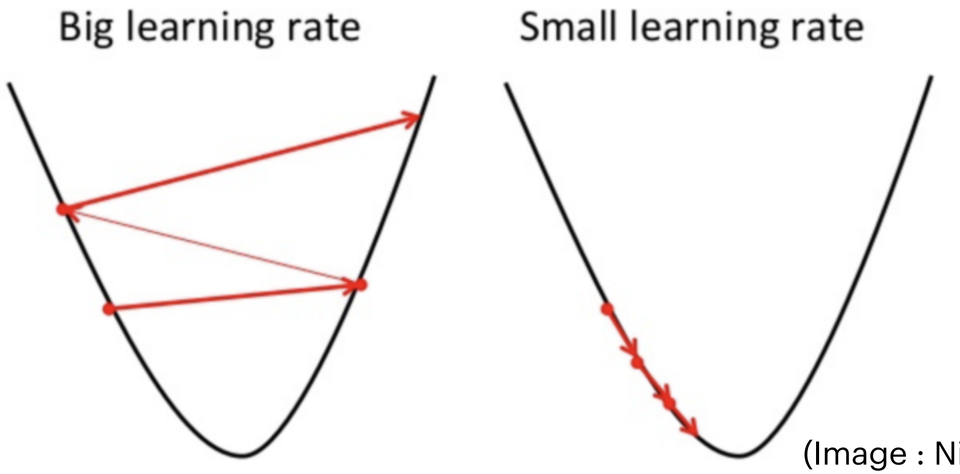
  - Stop when the derivative f' at the latest value Θ is sufficiently small, i.e.

$$|f'(\Theta^{(t)})| < \epsilon$$

Gradient Descent Convergence Theorem

- **Theorem**: If J is convex, then for any desired accuracy ε, there is some step-size η s.t. Gradient Descent will converge to within ε of the optimal Θ

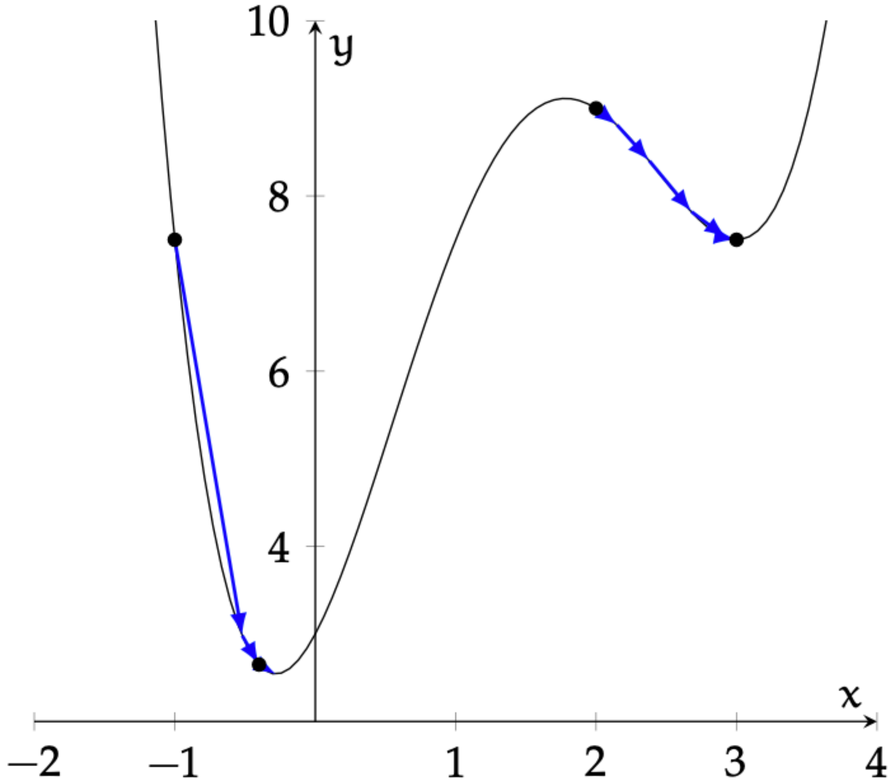
- However, we must be careful when choosing the step-size!
  - Bad step-size choice will lead to slow convergence, oscillation around the minimum, or divergence



(Image : Niklas Donges)

Non-Convex Case

- If J is non-convex, the convergence result depends on θ_{init}
- It may reach a local minimum, instead of a minimum of the function



(Image : MIT 6.036 Lecture Note)

Multiple Dimensions

- The extension to the case of multi-dimensional Θ is straightforward.
- Let's assume $\Theta \in \mathbb{R}^m$, so $J : \mathbb{R}^m \rightarrow \mathbb{R}$. The gradient of J w.r.t. Θ is

$$\nabla_\Theta J = \begin{bmatrix} \partial J / \partial \Theta_1 \\ \vdots \\ \partial J / \partial \Theta_m \end{bmatrix}$$

- The algorithm remain the same, except that the update step becomes:

$$\Theta^{(t)} = \Theta^{(t-1)} - \eta \nabla_\Theta J(\Theta^{(t-1)})$$

- And we need to change the termination criterion, for example

$$| f(\Theta^{(t)}) - f(\Theta^{(t-1)}) | < \epsilon$$

(Which is sensible no matter the dimensionality of Θ)

Application to logistic regression

- We'll see a simple example using logistic regression (lr)

$$J_{\text{lr}}(\theta, \theta_0) = \frac{1}{n} \sum_{i=1}^{n} \mathscr{L}_{\text{nll}}(g^{(i)}, y^{(i)}) + \frac{\lambda}{2} ||\theta||^2$$

nll: negative log likelihood

- We consider linear logistic classifier, and parameter is consisted of vector θ, and a scalar θ_0

Application to logistic regression

- Let $g^{(i)} = \sigma(\theta^\top \mathbf{x^{(i)}} + \theta_0)$, where $\sigma(\cdot)$ is a sigmoid function
- Then we can find the gradients respect to θ, θ_0 [try yourself]:

$$J_{\text{lr}}(\theta, \theta_0) = \frac{1}{n} \sum_{i=1}^{n} \mathscr{L}_{\text{nll}}(g^{(i)}, y^{(i)}) + \frac{\lambda}{2} \|\theta\|^2$$

$$\nabla_\theta J_{\text{lr}}(\theta, \theta_0) = \frac{1}{n} \sum_{i=1}^{n} \left(g^{(i)} - y^{(i)}\right) \mathbf{x^{(i)}} + \lambda\theta$$

$$\frac{\partial J_{\text{lr}}(\theta, \theta_0)}{\partial \theta_0} = \frac{1}{n} \sum_{i=1}^{n} \left(g^{(i)} - y^{(i)}\right)$$

Application to logistic regression

- Note that $\nabla_\theta J$ will be of shape $d \times 1$ and $\partial J / \partial \theta_0$ will be a scalar since we have separated θ_0 from θ in previous equations.
- **Study questions**:
  1. Convince yourself that the dimensions of all these quantities are correct, under the assumption that θ is $d \times 1$. How does d relates to m as discussed for Θ in the previous section?
  2. Compute $\nabla_\theta \|\theta\|^2$ by finding the vector of partial derivatives $(\partial \|\theta\|^2 / \partial \theta_1, \cdots, \partial \|\theta\|^2 / \partial \theta_d)$. What is the shape of $\nabla_\theta \|\theta\|^2$?
  3. Compute $\nabla_\theta \mathscr{L}_{\text{nll}}(\sigma(\theta^\top x + \theta_0), y)$ by finding the vector of partial derivatives $\partial \mathscr{L}_{\text{nll}}(\sigma(\theta^\top x + \theta_0), y) / \partial \theta_1, \cdots, \mathscr{L}_{\text{nll}}(\sigma(\theta^\top x + \theta_0), y) / \partial \theta_d$
  4. Use these last two results to verify our derivation above

Application to Logistic Regression

- Putting everything together, our Gradient Descent Algorithm for LR becomes: [try yourself]

Algorithm 15: LR-Gradient-Descent$(\theta_{\text{init}}, \theta_{0\text{init}}, \eta, \epsilon)$

$\theta^{(0)} = \theta_{\text{init}}$

$\theta_0^{(0)} = \theta_{0\text{init}}$

$t = 0$

while $\left| J_{lr}(\theta^{(t)}, \theta_0^{(t)}) - J_{lr}(\theta^{(t-1)}, \theta_0^{(t-1)}) \right| > \epsilon$ **do**

  $t = t + 1$

  $\theta^{(t)} =$

  $\theta^{(t-1)} - \eta \left(\frac{1}{n} \sum_{i=1}^{n} \left(\sigma \left({\theta^{(t-1)}}^{\top} x^{(i)} + \theta_0^{(t-1)} \right) - y^{(i)} \right) x^{(i)} + \lambda \theta^{(t-1)} \right)$

  $\theta^{(t)} = \theta^{(t-1)} - \eta \left(\frac{1}{n} \sum_{i=1}^{n} \left(\sigma \left({\theta^{(t-1)}}^{\top} x^{(i)} + \theta_0^{(t-1)} \right) - y^{(i)} \right) \right)$

return $\theta^{(t)}, \theta_0^{(t)}$

- It is just a mixture of a Gradient Descent and LR

Stochastic Gradient Descent

- Using (normal) Gradient descent always leads to the same output when the initial point is the same
- So we can add a little '**noise**' by randomly picking a single data point, and it still goes to the gradient direction in expectation

Stochastic Gradient Descent

- Let's assume our objective has the form of $f(\mathbf{\Theta}) = \sum_{i=1}^{n} f_i(\mathbf{\Theta})$

Algorithm 7: Stochastic-Gradient-Descent$(\mathbf{\Theta}_{\text{init}}, \eta, f, \nabla_{\Theta} f_1, \cdots, \nabla_{\Theta} f_n, T)$

$\mathbf{\Theta}^{(0)} = \mathbf{\Theta}_{\text{init}}$
for $t = 1$ *to* T **do**
  randomly select $i \in \{1, 2, \cdots, n\}$
  $\mathbf{\Theta}^{(t)} = \mathbf{\Theta}^{(t-1)} - \eta(t) \nabla_{\Theta} f_i(\mathbf{\Theta}^{(t-1)})$
  return $\mathbf{\Theta}^{(t)}$

- Note that η is indexed by the iteration of the algorithm, t. (See next slide)
- Finding a good 'stopping criterion' is a tricky problem for SGD:
  - Here we've just chosen to stop after a fixed number of iterations T.

Stochastic Gradient Descent: Theorem

ADVANCED

- For SGD to converge to a local optimum as t increases, the stepsize $\eta(t)$ has to decrease
- **Theorem**: If J is convex, and $\eta(t)$ is a sequence satisfying

$$\sum_{t=1}^{\infty} \eta(t) = \infty \text{ and } \sum_{t=1}^{\infty} \eta(t)^2 < \infty,$$

- Then SGD **converges** with probability 1.0 to the optimal Θ!

Some Intuitions: Why SGD might be better?

- If f is actually non-convex and has many shallow local optima, taking samples from the gradient at some point Θ might "bounce" you around the landscape and out of the local optima

- Sometimes regular GD might overfit the training set: SGD might result in lower test error

- Regular GD requires computing over every datapoint, thus SGD is useful in runtime and memory savings

Stochastic Gradient Descent (SGD)

- Let's use $\mathbf{W}$ as a general case of the model parameters, and L as a loss function.
- Update weights for **each sample**
  - + Fast, online
  - − Sensitive to noise

$$L = \frac{1}{2}(y^n - \hat{y}^n)^2 \qquad w_i(t+1) = w_i(t) - \eta \frac{\partial L^n}{\partial w_i}$$

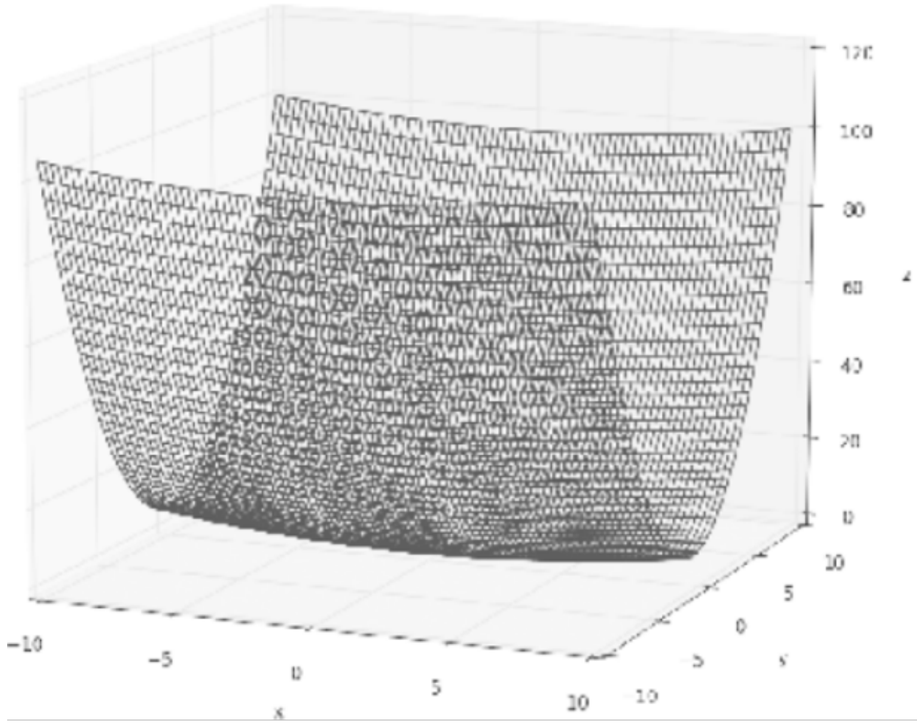
- Minibatch SGD: Update weights for **a small set of samples**

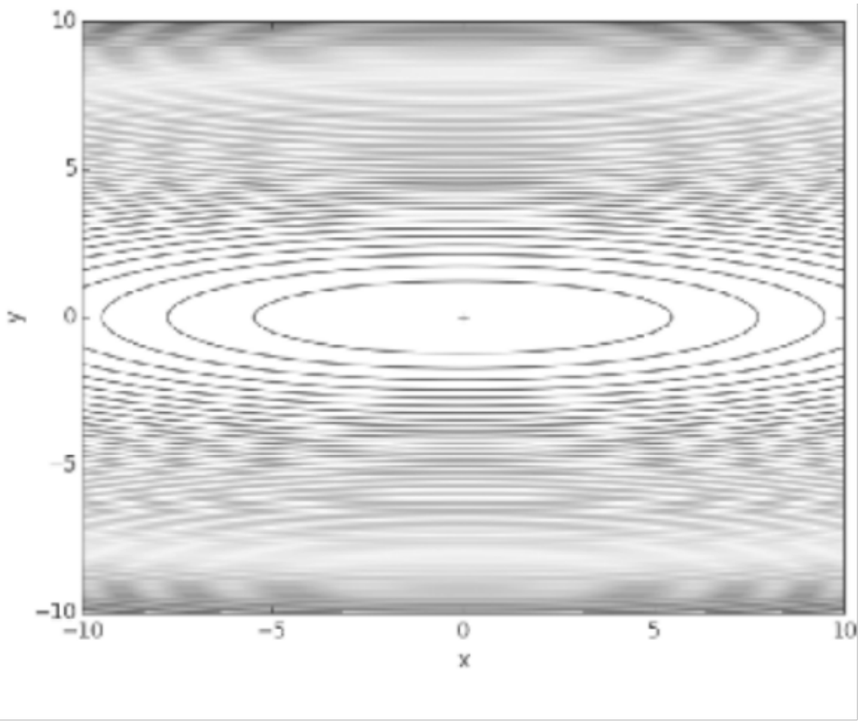
$$L = \frac{1}{2}\sum_{n \in B}(y^n - \hat{y}^n)^2 \qquad w_i(t+1) = w_i(t) - \eta \frac{\partial L^B}{\partial w_i}$$

  - + Fast, online
  - + Robust to noise

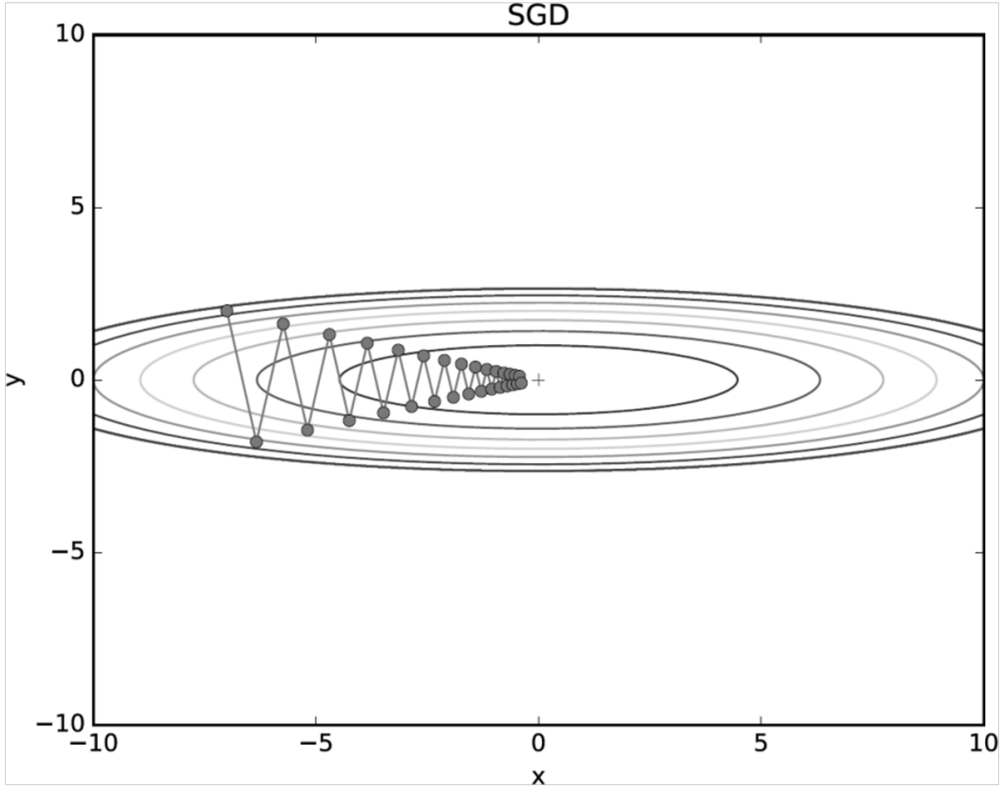
Stochastic Gradient Descent (SGD)

- $f(x, y) = \frac{1}{20}x^2 + y^2$



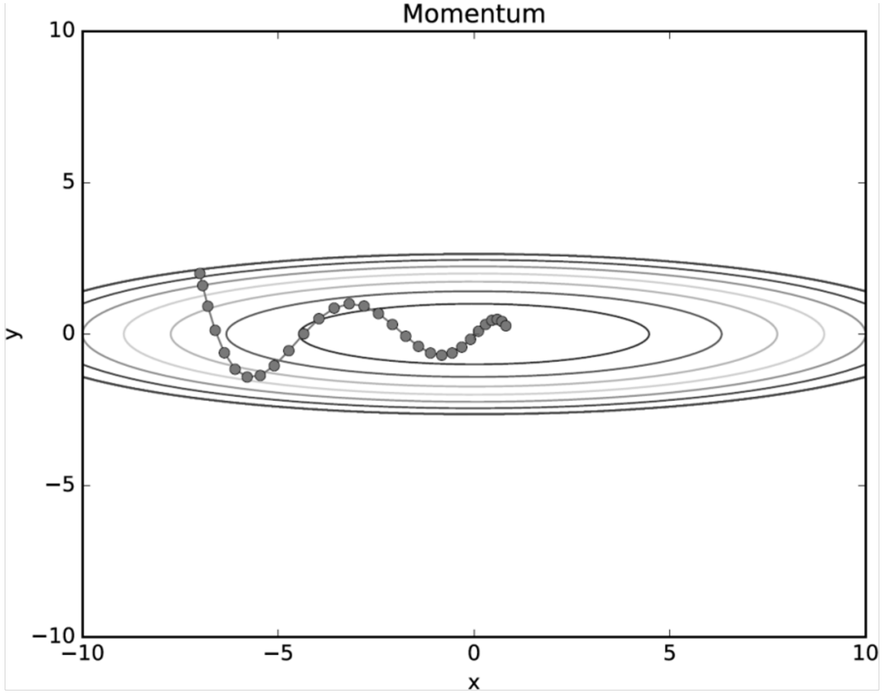


Stochastic Gradient Descent (SGD)



Reducing oscillation? Momentum, AdaGrad, Adam

Momentum



+ Converge faster + Avoid oscillation

$$v_i(t) = \alpha v_i(t-1) - \eta \frac{\partial L}{\partial w_i}(t)$$

$$\mathbf{w}(t+1) = \mathbf{w}(t) + \mathbf{v}(t)$$

AdaGrad & Adam

- AdaGrad Optimizer: Learning rates are adaptively determined for each parameter

$$\mathbf{h} \leftarrow \mathbf{h} + \frac{\partial L}{\partial \mathbf{W}} \odot \frac{\partial L}{\partial \mathbf{W}}$$

$$\mathbf{W} \leftarrow \mathbf{W} - \frac{\eta}{\sqrt{\mathbf{h} + \epsilon}} \frac{\partial L}{\partial \mathbf{W}}$$

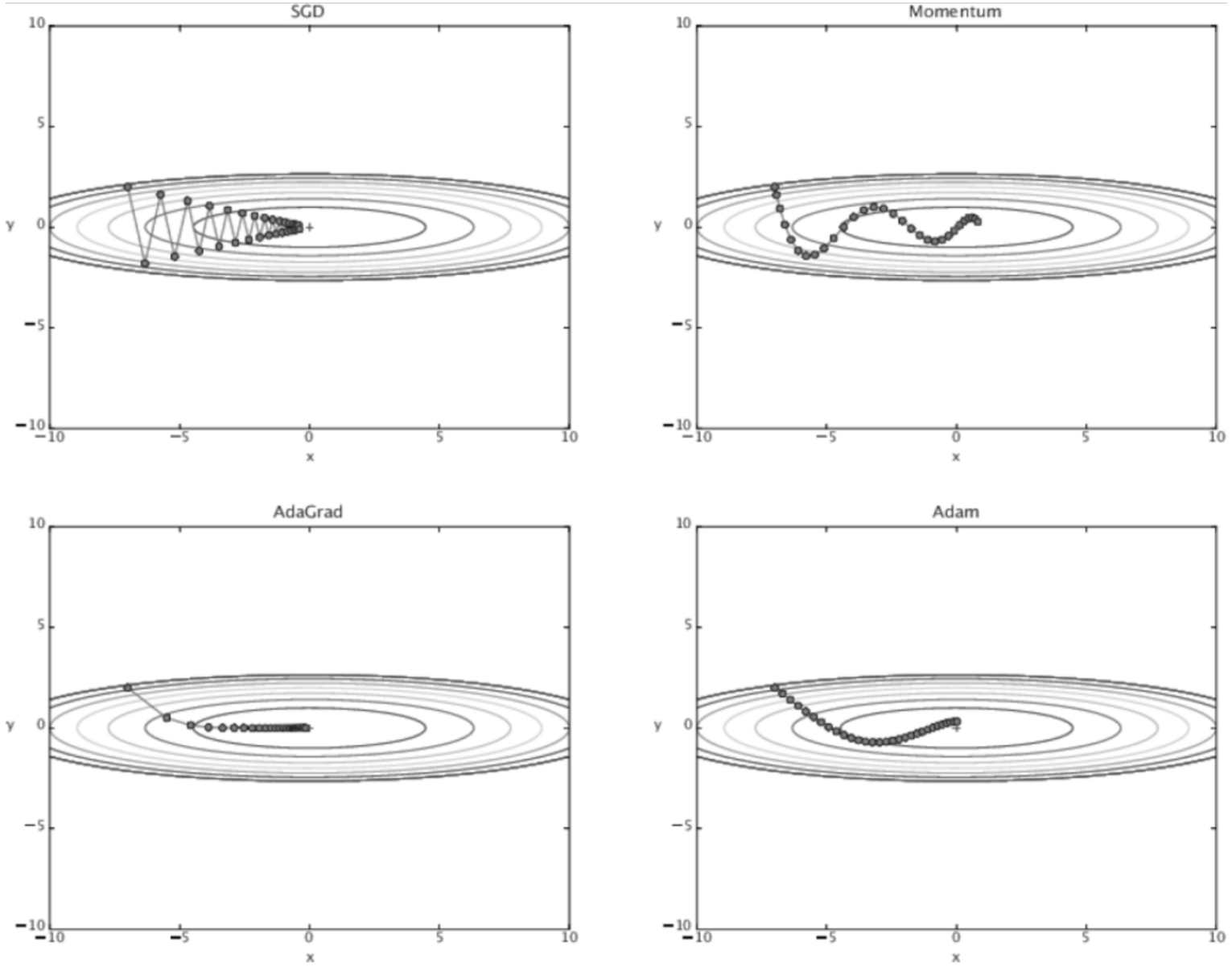
- Adam Optimizer ~ Moment + AdaGrad
  - Adam: A Method for Stochastic Optimization (242,361 citations as of Apr 29, 2025)
  - Simple idea, huge impacts (See next slide)

Adam Optimizer - from the paper

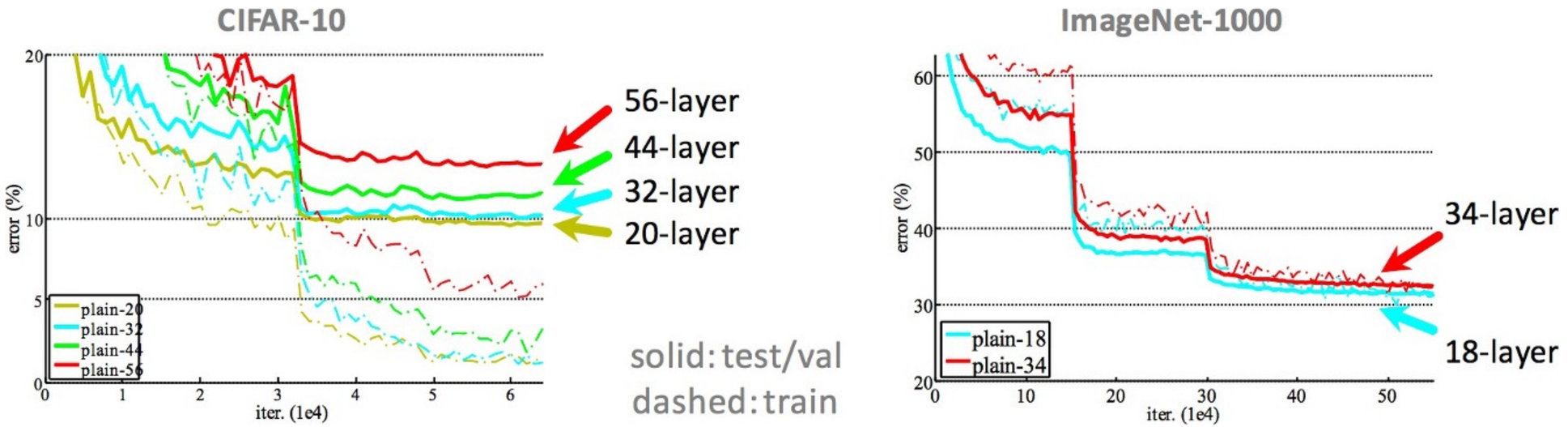
Algorithm 1: *Adam*, our proposed algorithm for stochastic optimization. See section 2 for details, and for a slightly more efficient (but less clear) order of computation. g_t^2 indicates the elementwise square $g_t \odot g_t$. Good default settings for the tested machine learning problems are $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$. All operations on vectors are element-wise. With β_1^t and β_2^t we denote β_1 and β_2 to the power t.

Require: α: Stepsize
Require: $\beta_1, \beta_2 \in [0, 1)$: Exponential decay rates for the moment estimates
Require: $f(\theta)$: Stochastic objective function with parameters θ
Require: θ_0: Initial parameter vector
$m_0 \leftarrow 0$ (Initialize 1st moment vector)
$v_0 \leftarrow 0$ (Initialize 2nd moment vector)
$t \leftarrow 0$ (Initialize timestep)
while θ_t not converged **do**
$t \leftarrow t + 1$
$g_t \leftarrow \nabla_\theta f_t(\theta_{t-1})$ (Get gradients w.r.t. stochastic objective at timestep t)
$m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ (Update biased first moment estimate)
$v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ (Update biased second raw moment estimate)
$\widehat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ (Compute bias-corrected first moment estimate)
$\widehat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ (Compute bias-corrected second raw moment estimate)
$\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \widehat{m}_t / (\sqrt{\widehat{v}_t} + \epsilon)$ (Update parameters)
end while
return θ_t (Resulting parameters)

Comparison

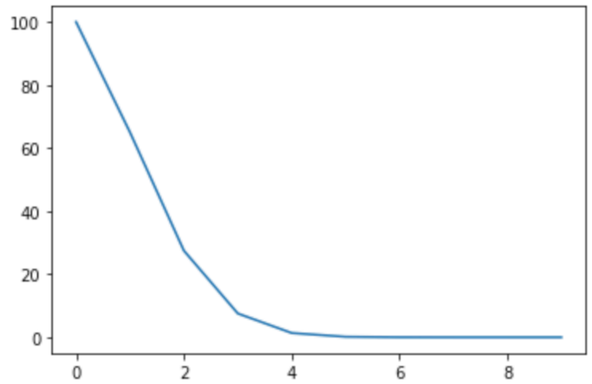


Examples

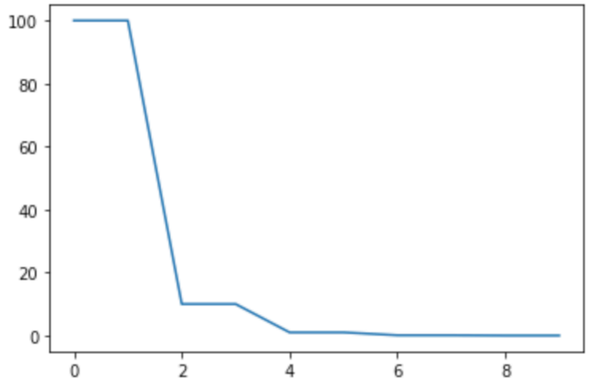


- ResNet training curve
  - Study Question: can you explain why there is a steps in the curve?

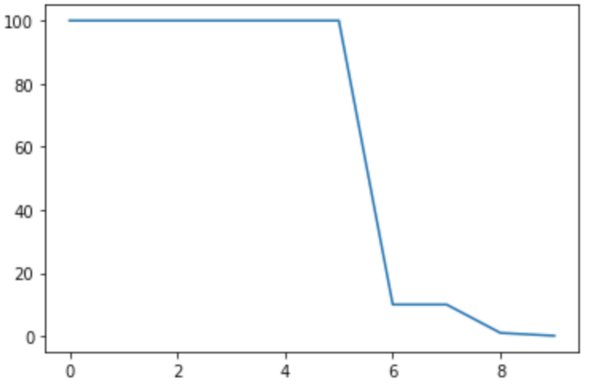
Additional LR schedulers



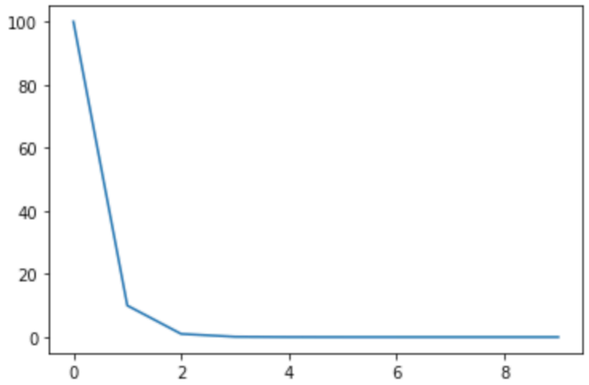
MultiplicativeLR



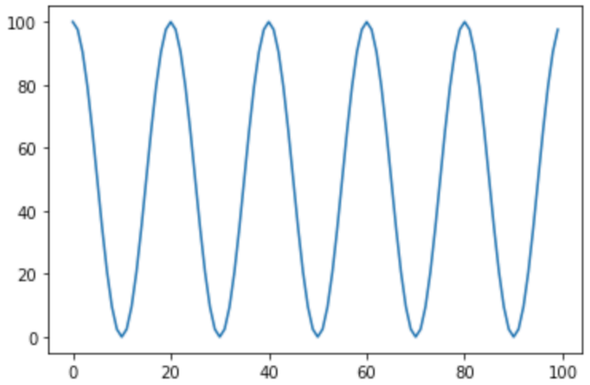
StepLR



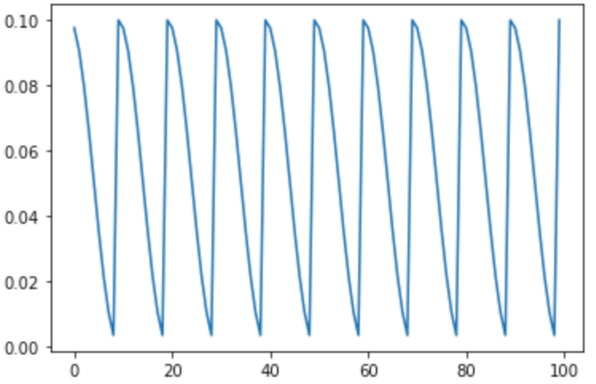
MultiStepLR



ExponentialLR



CosineAnnealingLR



CosineAnnealingWarmRestarts

Interesting to read: SGDR: Stochastic Gradient Descent with Warm Restarts. https://arxiv.org/abs/1608.03983

https://www.kaggle.com/code/isbhargav/guide-to-pytorch-learning-rate-scheduling

QnA

- Any questions?

- Note: this lecture notes are largely adopted from
  - MIT 6.036(2019 Fall), Stanford CS229(2023-2024 Summer)